



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

Вариант 676
по дисциплине

«Распределённые системы хранения данных»:

Выполнили: Ястребов-Амирханов Алекси, Исупов Никита

Группа: Р3332/Р3331

Преподаватель: Афанасьев Дмитрий Борисович

Санкт-Петербург,

2026

Оглавление

Задание.....	2
Этап 1. Резервное копирование.	3
Этап 2. Потеря основного узла.	6
Этап 3. Повреждение файлов БД	8
Этап 4. Логическое повреждение данных.....	10
Вывод	13

Задание.

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:

Периодические обособленные (standalone) полные копии.

Полное резервное копирование (pg_basebackup) по расписанию (cron) два раза в сутки. Необходимые файлы WAL должны быть в составе полной копии, отдельно их не архивировать. Срок хранения копий на основной системе - 1 неделя, на резервной - 1 месяц. По истечении срока хранения, старые архивы должны автоматически уничтожаться.

- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
 - Средний объем новых данных в БД за сутки: 300МБ.
 - Средний объем измененных данных за сутки: 350МБ.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
 - удалить с диска директорию конфигурационных файлов СУБД со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.
- Выполнить восстановление данных из резервной копии, учитывая следующее условие:

- исходное расположение директории PGDATA недоступно - разместить данные в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Генерация файла на резервном узле с помощью `pg_dump` и последующее применение файла на основном узле.

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
 - в любой таблице с внешними ключами подменить значения ключей на случайные (INSERT, UPDATE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

Этап 1. Резервное копирование.

Для автоматического копирования без ввода пароля настраивается аутентификация по ключам.

```
# на основном сервере (pg197)
ssh-keygen -N ""
ssh-copy-id postgres0@pg196
```

Создан скрипт `~/pg_backup_full.sh`, который выполняет:

- создание локальной копии с помощью `pg_basebackup` (формат tar);
- копирование полученного архива на резервный сервер;
- удаление старых копий согласно политике хранения.

```
#!/usr/bin/env bash

DATE=$(date +%Y-%m-%d_%H-%M)
BACKUP_DIR="/var/db/postgres0/base_backups/$DATE"
REMOTE_HOST="postgres0@pg196"
REMOTE_BACKUP_DIR="/var/db/postgres0/base_backups"

# Создание родительских директорий
mkdir -p /var/db/postgres0/base_backups
ssh $REMOTE_HOST "mkdir -p $REMOTE_BACKUP_DIR"
```

```

# Удаление предыдущей копии за эту дату (если вдруг осталась)
if [ -d "$BACKUP_DIR" ]; then
rm -rf "$BACKUP_DIR"
fi

# Создание полной копии (standalone) с включением всех файлов WAL
echo "Starting backup to $BACKUP_DIR"
pg_basebackup -D "$BACKUP_DIR" -U postgres0 -p 9706 -h pg197 -Ft -z -X stream -P

# Копирование на резервный сервер
echo "Copying backup to remote server"
scp -r "$BACKUP_DIR" "$REMOTE_HOST:$REMOTE_BACKUP_DIR/"

# Очистка локальных копий старше 7 дней
echo "Cleaning up old backups on local server"
find /var/db/postgres0/base_backups -type d -mtime +7 -exec rm -rf {} \; 2>/dev/null || true

# Очистка удалённых копий старше 30 дней
echo "Cleaning up old backups on remote server"
ssh $REMOTE_HOST "find $REMOTE_BACKUP_DIR -type d -mtime +30 -exec rm -rf {} \; 2>/dev/null || true"

echo "Backup completed successfully"

```

Для автоматического запуска дважды в сутки (в 00:00 и 12:00) в планировщик добавлена задача:

```
crontab -e
```

Добавлена строка:

```
0 0,12 * * * ~/pg_backup_full >> ~/backup.log 2>&1
```

Расчёт объёма резервных копий через месяц

- Средний объём новых данных в БД за сутки: **300 МБ** (вставки, увеличивающие физический размер).
- Средний объём изменённых данных за сутки: **350 МБ** (обновления, которые в модели не увеличивают размер БД).
- Частота бэкапов: **2 раза в сутки**.
- Политика хранения:
 - основной сервер – 7 дней (14 последних копий);
 - резервный сервер – 30 дней (60 последних копий).
- Начальный размер базы в момент первого бэкапа условно принят равным **0 МБ**.

- Рост базы линейный: каждые сутки добавляется 300 МБ, значит между двумя последовательными бэкапами (с интервалом 12 часов) размер увеличивается на **150 МБ**.

Как меняется размер каждой копии

Обозначим копии номерами $k = 0, 1, 2, \dots, 59$ (за месяц их будет 60).

Размер S_k копии с номером k равен

$$S_k = 150 \cdot k \text{ (MB)}$$

Объём на резервном узле через месяц

На резервном узле хранятся все 60 копий. Их сумма – это сумма арифметической прогрессии:

$$V_{\text{Резервный}} = \frac{(S_0 + S_{59}) \cdot 60}{2} = \frac{(0 + 8850) \cdot 60}{2} = 265500 \text{ MB} \approx 259,3 \text{ GB}$$

Объём на основном узле (последние 7 дней)

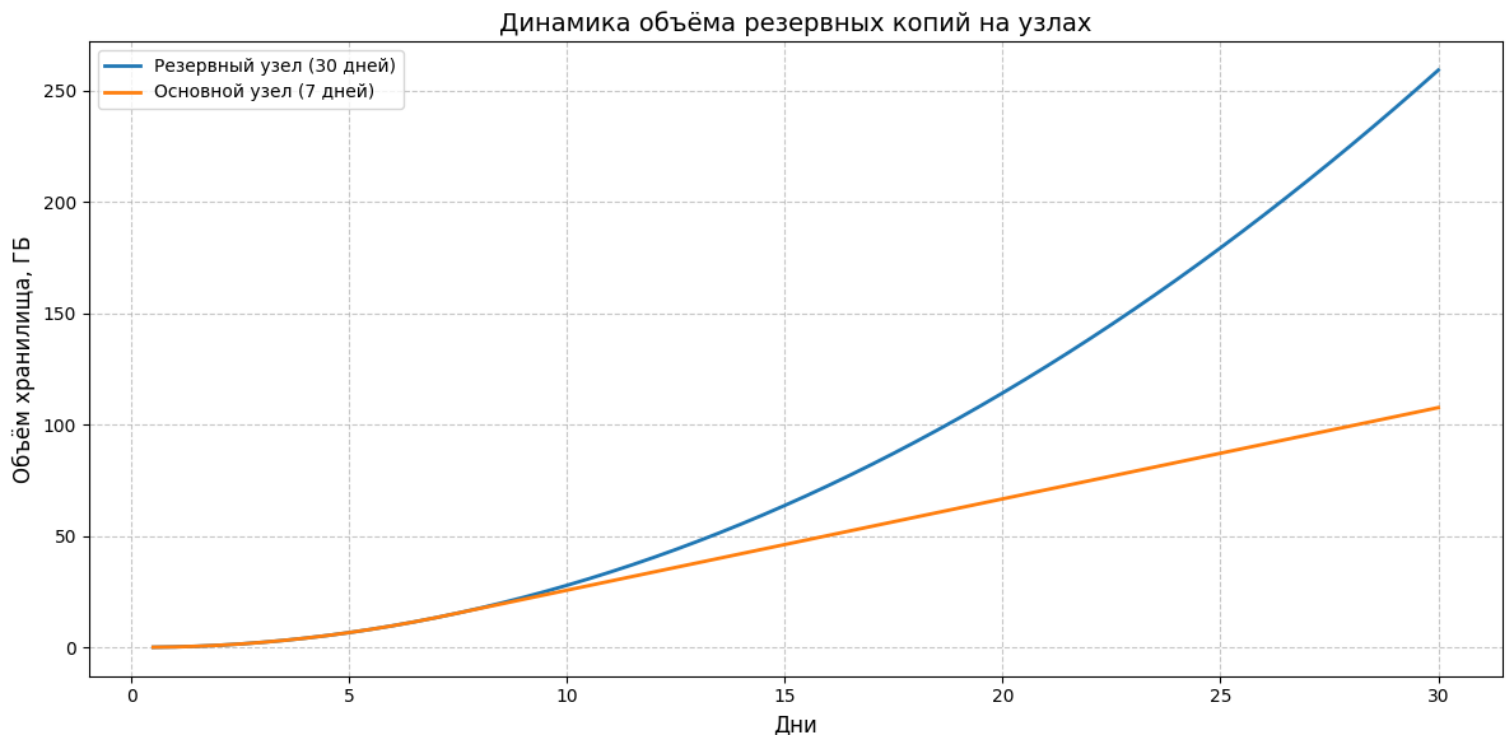
За последние 7 дней сделано 14 копий – с номерами от $k = 46$ до $k = 59$. Их размеры:

$$S_{46} = 150 \cdot 46 = 6900 \text{ (MB)}$$

$$S_{59} = 150 \cdot 59 = 8850 \text{ (MB)}$$

Сумма:

$$V_{\text{Основной}} = \frac{(6900 + 8850) \cdot 14}{2} = 15750 \cdot 7 = 110250 \text{ MB} \approx 107,7 \text{ GB}$$



Если учитывать сжатие

В скрипте используется сжатие (`pg_basebackup -z`). Ориентировочный коэффициент сжатия – **0,7**. Тогда реальные объёмы будут примерно:

- резервный узел: $265\,500 \times 0,7 \approx 185\,850 \text{ MB} \approx 181,5 \text{ ГБ}$
- основной узел: $110\,250 \times 0,7 \approx 77\,175 \text{ MB} \approx 75,4 \text{ ГБ}$

Этап 2. Потеря основного узла.

Основной узел: pg197, пользователь postgres0.

Резервный узел: pg196, пользователь postgres0.

Остановка работающего экземпляра PostgreSQL на резервном узле:

```
pg_ctl -D /var/db/postgres0/wml19 -l /var/db/postgres0/wml19/server.log stop
```

```
#!/usr/bin/env bash
set -euo pipefail # Прерывать выполнение при любой ошибке, неопределённой переменной
или ошибке в конвейере

# Настройка переменных окружения
RECOVERY_DIR="/var/db/postgres0/wml19" # Куда распаковывать base/pg_wal
TABLESPACE_ROOT="/var/db/postgres0" # Корень, куда будут распакованы архивы
вида 16388.tar.gz -> /var/db/postgres0/16388
BACKUP_BASE_DIR="/var/db/postgres0/base_backups"

# Создание необходимых каталогов
mkdir -p "$RECOVERY_DIR"
mkdir -p "$TABLESPACE_ROOT"

# Поиск последней резервной копии.
LATEST_BACKUP_DIR=$(find "$BACKUP_BASE_DIR" -mindepth 1 -maxdepth 1 -type d |
sort | tail -n 1 || true)

# find выдаёт все подкаталоги первого уровня внутри BACKUP_BASE_DIR.
# Благодаря формату имён YYYY-MM-DD сортировка по алфавиту эквивалентна
хронологической.
# tail -n 1 берёт последний (самый свежий) каталог.
# Если каталогов нет, переменная останется пустой.
if [ -z "$LATEST_BACKUP_DIR" ]; then
    echo "ERROR: No backup directories found in $BACKUP_BASE_DIR"
    exit 1
fi

echo "Using backup directory: $LATEST_BACKUP_DIR"

# Распаковка архивов с учётом их назначения
short -s nullglob # Позволяет циклу не выполняться, если нет файлов, соответствующих
шаблону
```

```

for tarfile in "$LATEST_BACKUP_DIR"/*.tar "$LATEST_BACKUP_DIR"/*.tar.gz
"$LATEST_BACKUP_DIR"/*.tgz; do
    [ -f "$tarfile" ] || continue
    name=$(basename "$tarfile")

    # Если имя полностью числовое (OID.tar.gz), распаковываем в /var/db/postgres0/<oid>/
    if [[ "$name" =~ ^([0-9]+)\.tar(\.gz)?$ ]] || [[ "$name" =~ ^([0-9]+)\.tgz$ ]]; then
        if [[ "$name" =~ ^([0-9]+) ]]; then
            oid="{BASH_REMATCH[1]}"
        else
            echo "Skipping unexpected numeric-named file: $name"
            continue
        fi

        dest="$TABLESPACE_ROOT/$oid"
        echo "Extracting tablespace archive $name -> $dest"
        mkdir -p "$dest"

        if [[ "$tarfile" == *.gz ]] || [[ "$tarfile" == *.tgz ]]; then
            tar -xzf "$tarfile" -C "$dest"
        else
            tar -xf "$tarfile" -C "$dest"
        fi
    else
        # Для остальных (base.tar.gz, pg_wal.tar.gz и т.п.) — распакуем в RECOVERY_DIR
        echo "Unpacking $name -> $RECOVERY_DIR"
        mkdir -p "$RECOVERY_DIR"

        if [[ "$tarfile" == *.gz ]] || [[ "$tarfile" == *.tgz ]]; then
            tar -xzf "$tarfile" -C "$RECOVERY_DIR"
        else
            tar -xf "$tarfile" -C "$RECOVERY_DIR"
        fi
    fi
done

shopt -u nullglob # Возвращаем стандартное поведение

# Копирование несжатых файлов
if [ -d "$LATEST_BACKUP_DIR/base" ] || [ -d "$LATEST_BACKUP_DIR/global" ]; then
    echo "Copying uncompressed files from backup directory to $RECOVERY_DIR"
    cp -r "$LATEST_BACKUP_DIR"/* "$RECOVERY_DIR"/ || true
fi

# Установка прав владельца
if [ "$(id -u)" -eq 0 ]; then
    echo "Setting ownership to postgres:postgres for recovery and tablespace dirs (if possible)..."
    chown -R postgres:postgres "$RECOVERY_DIR" || true
    chown -R postgres:postgres "$TABLESPACE_ROOT" || true
fi

```

```
echo "Recovery completed."
echo " RECOVERY_DIR = $RECOVERY_DIR"
echo " Tablespaces under = $TABLESPACE_ROOT"
```

Создаем файл recovery.signal и настраиваем конфигурацию postgresql.conf

```
#recovery.signal
touch /var/db/postgres0/wml19/recovery.signal

# postgresql.conf
echo "restore_command = 'cp /var/db/postgres0/wml19/%f %p'" >> ~/wml19/postgresql.conf
```

Теперь нужно восстановить табличные пространства. Сначала нужно узнать исходное название директорий, в которых находились табличные пространства. Для этого нужно посмотреть содержимое файла tablespace_map

```
cat wml19/tablespace_map

>16389 /var/db/postgres0/ahq84
>16388 /var/db/postgres0/kbs76
```

Теперь переименуем директории табличных пространств и создадим символические ссылки в wml19/pg_tblspc

```
mv 16388 kbs76
mv 16389 ahq84

ln -s /var/db/postgres0/ahq84 /var/db/postgres0/wml19/pg_tblspc/16389
ln -s /var/db/postgres0/kbs76 /var/db/postgres0/wml19/pg_tblspc/16388
```

Используем утилиту pg_ctl для запуска экземпляра PostgreSQL с указанием новой директории данных и необходимых параметров.

```
pg_ctl -D /var/db/postgres0/wml19 -o "-p 9706" start
```

Этап 3. Повреждение файлов БД

Для имитации повреждения файлов БД удаляем директорию с данными и конфигурацией PostgreSQL:

```
rm -rf /var/db/postgres0/wml19
```


Пытаемся запустить сервер:

```
pg_ctl -D wml19 start  
  
> pg_ctl: каталог "wml19" не существует
```

Пытаемся подключиться к работающему серверу:

```
psql -h localhost -p 9706 -U postgres0 -d postgres  
  
>psql: ошибка: подключиться к серверу "localhost" (::1), порту 9706 не удалось: Connection refused  
> Сервер действительно работает по данному адресу и принимает TCP-соединения?  
>подключиться к серверу "localhost" (127.0.0.1), порту 9706 не удалось: Connection refused  
> Сервер действительно работает по данному адресу и принимает TCP-соединения?
```

Подготовка новой директории для восстановленных данных

```
mkdir -p /var/db/postgres0/wml19  
chmod 700 /var/db/postgres0/wml19
```

Определение последней резервной копии

```
LATEST_BACKUP=$(ls -1d /var/db/postgres0/base_backups/* | sort | tail -1)  
echo "Using backup: $LATEST_BACKUP"
```

Распаковка архивов резервной копии

Резервная копия, созданная с помощью pg_basebackup -Ft -z, представляет собой один или несколько .tar.gz файлов. Распаковываем их непосредственно в целевую директорию.

```
for tarfile in "$LATEST_BACKUP"/*.tar.gz; do  
    if [ -f "$tarfile" ]; then  
        tar -xzf "$tarfile" -C /var/db/postgres0/wml19  
    fi  
done
```

Восстановление ссылок на табличные пространства

```
ln -sf /var/db/postgres0/ahq84 /var/db/postgres0/wml19/pg_tblspc/16389  
ln -sf /var/db/postgres0/kbs76 /var/db/postgres0/wml19/pg_tblspc/16388
```

```
touch ~/wml19/recovery.signal

pg_ctl -D /var/db/postgres0/wml19 -o "-p 9706" start
```

Этап 4. Логическое повреждение данных.

Подготовка тестовых данных

```
-- Создание таблиц на основном узле (pg197)
CREATE TABLE table1 (
  id SERIAL PRIMARY KEY,
  text1 TEXT
);

CREATE TABLE table2 (
  id INTEGER,
  text2 TEXT,
  CONSTRAINT fk_table1
    FOREIGN KEY (id)
    REFERENCES table1(id)
);

-- Заполнение данными
INSERT INTO table1 (text1) VALUES
('text1'),
('text2'),
('text3');

INSERT INTO table2 (id, text2) VALUES
(1, 'ref1'),
(2, 'ref2'),
(3, 'ref3');

-- Проверка целостности
SELECT t1.id, t1.text1, t2.text2
FROM table1 t1
JOIN table2 t2 ON t1.id = t2.id;
```

Результат запроса

id	text1	text2
1	text1	ref1
2	text2	ref2
3	text3	ref3

Создание резервной копии до возникновения ошибки на резервном узле

```
pg_dump -h pg197 -p 9706 -U postgres0 -d uglyorangebird > backup.sql
```

Симуляция логического повреждения

Имитируем ошибочную операцию: обновим внешний ключ в одной из строк таблицы table2, установив значение, отсутствующее в table1.

```
-- Отключаем триггеры внешнего ключа на table2
ALTER TABLE table2 DISABLE TRIGGER ALL;

-- Ошибочное обновление: меняем id с 1 на 999 (такого id в table1 нет)
UPDATE table2
SET id = 999
WHERE id = 1;

-- Включаем триггеры обратно (для полноты картины, но после ошибки они уже не помогут)
ALTER TABLE table2 ENABLE TRIGGER ALL;
```

После этой операции данные становятся логически повреждёнными. В таблице table2 появилась запись с id = 999, которая не ссылается ни на одну строку в table1.

Проверим результат повреждения

```
-- Проверка с JOIN (запись с id=1 из table2 пропала, вместо неё id=999)
SELECT t1.id, t1.text1, t2.text2
FROM table1 t1
RIGHT JOIN table2 t2 ON t1.id = t2.id;
```

Результат:

id	text1	text2
2	text2	ref2
3	text3	ref3
		ref1

< строка с id=999, не имеющая соответствия в table1

Восстановление данных из резервной копии

```
-- На основном узле
dropdb -h pg197 -p 9706 -U postgres0 uglyorangebird
createdb -h pg197 -p 9706 -U postgres0 uglyorangebird
```

-- На резервном узле

```
psql -h pg197 -p 9706 -U postgres0 -d uglyorangebird < backup.sql
```

Вывод

В ходе выполнения работы были реализованы четыре сценария обеспечения отказоустойчивости и восстановления PostgreSQL:

Настроено регулярное создание полных резервных копий дважды в сутки с помощью pg_basebackup, с автоматической ротацией (7 дней на основном узле, 30 дней на резервном). Произведён расчёт ожидаемого объёма хранилища, который составит ≈ 259 ГБ на резервном узле и ≈ 108 ГБ на основном (без учёта сжатия), что позволяет планировать дисковые ресурсы.

При полной потере основного узла восстановление выполнено на резервном узле с использованием разработанного скрипта. Скрипт автоматически находит последнюю копию, распаковывает основные файлы и табличные пространства в соответствующие каталоги, корректирует права доступа. После ручной настройки символических ссылок на табличные пространства сервер успешно запущен, данные доступны.

При повреждении файлов БД на основном узле (симуляция удаления PGDATA) восстановление произведено из локальной копии в новую директорию.

При логическом повреждении данных (нарушение ссылочной целостности) использован логический дамп созданный до инцидента. После очистки повреждённых таблиц дамп применён, что вернуло базу в корректное состояние.

Все этапы подтвердили работоспособность выбранных стратегий резервного копирования и восстановления. Система готова к различным сбоям: от потери узла до ошибочных действий пользователя, обеспечивая минимальное время простоя и сохранность данных.